Proof of Correctness And Running time Analysis for Question 3

```
Rb_enumerate(x, a, b):
    if (x == NIL):
        return
    if (x.key > a):
        Rb_enumerate(x.left, a, b)
    if (a <= x.key <= b):
        Print x.key
    if (x.key < b):
        Rb_enumerate(x.right, a, b)
```

Proof of Correctness

~~It print every Key K such that a≤K≤b~~

1] Any Key printed by Rb_enumerate satisfies $a \le Key(K) \le b$.

↓Proof

The print statement print Key(x) is executed if and only if the guard condition a<=Key(x) and Key(x)<=b evaluates to true. Thus, no elements outside the range [a, b] can be printed.

2] every node v in the subtree rooted at x whose key satisfies $a \le key(v) \le b$ will be visited and printed.

↓Proof (by using binary search property)

Suppose there exists a node v with $a \le key(v) \le b$ that is not visited. This can only happen if at some ancestor u of v, the recursive call towards v was skipped:-

a] If v lies in left(u), the recursive call is skipped only if $key(u) \le a$. But since $v \in left(u)$, we have $key(u) \le a$, which contradicts the assumption that $key(v) \ge a$.

$key(v) < key(u) \le a$.

similarly

b] If v lies right (u), the recursive call is skipped only if $key(u) \geq b$. but since $v \in right(u)$, we have $key(v) \Rightarrow key(u) \geq b$, which contradicts the assumption that $key(v) \leq b$.

$\Rightarrow$ hence no branch containing a valid target key $v \in [a,b)$ is ever missed. every key in $[a,b]$ is visited and printed.

3] Sorted order

↓

Since the algorithm traverses subtrees in the standard in-order sequence (left subtree → current node → right subtree), the keys in $[a,b]$ are printed in strictly increasing (sorted) order.

Running-time analysis

n = total no. of keys in Red-black tree

r = number of keys reported in the interval $[a,b]$

h = height of Red-black tree

↓

Since a Red-black tree is height-balanced

$$h = O(\log(n+1))$$

Visited Node

Reporting Node :- Nodes u such that $key(u) \in [a,b]$

* There are exactly r such nodes, each produces 1 output

Boundary Nodes :- Nodes u such that $key(u) \notin [a,b]$

* A node u with $key(u) < a$ is visited only if its parent has a key $> a$. Thus, such nodes lies on the search path from the root to the predecessor of a.
* Similarly, for $key(u) > b$, lies to the successor of b.

⇒ search path from the root down to a and b each have maximum length proportional to the height of the tree

$$h \leq 2 \log_2 (n+1) = O(\log(n+1))$$

⇒ The boundary nodes outside [a, b] form at most two path from the root down to the range boundaries. hence the total number of nodes visited is bounded by

$$O(h) = O(\log(n+1))$$

⇒ at each visited node, the algo. perform O(1) comparisons and constant work.

⇒ Total time complexity :-

Visited Nodes = Reporting Node + Boundary Nodes

$$= \gamma + O(\log(n+1))$$

$$= O(\gamma + \log(n+1))$$